



PhD Admission System

Periyar University — Complete System Workflow

8

PHASES

2

PROJECTS

40+

API ENDPOINTS

9

DB TABLES

STUDENT PORTAL
localhost:5173

ADMIN PANEL
localhost:5174

STUDENT API
localhost:5000

ADMIN API
localhost:5001



System Architecture Overview

Two independent React frontends, two Express backends, one shared MySQL database

Student Portal

Frontend (React/Vite) localhost:5173

Backend (Node/Express) localhost:5000

Authentication JWT Bearer Token

State Management Zustand Store

Pages Register, Login, Dashboard, Apply, Hall Ticket, Counselling, Payment

Admin Panel

Frontend (React/Vite) localhost:5174

Backend (Node/Express) localhost:5001

Authentication JWT + isAdmin Middleware

Default Login admin@periyar.edu / admin123

Pages Dashboard, Applications, Sessions, Hall Tickets, Counselling, Settings, Dropdowns, Community Fees, Uploads, Locations

Shared Database — MySQL 8+



Database: **rsm_db** | Both backends connect to the same MySQL instance | Tables: sessions, users, applications, hall_tickets, counselling_settings, counselling_applications, counselling_research_choices, community_fees, and master dropdown tables



8-Phase Admission Workflow at a Glance

Sequential gating — each phase unlocks only after the previous is complete



Phase 1
Session Setup



Phase 2
Student Register



Phase 3
Application Form



Phase 4
Admin Review



Phase 5
Fee Payment



Phase 6
Hall Ticket



Phase 7
Exam & Results



Phase 8
Counselling



Session-based architecture: Every student is tied to an active session (e.g. "July 2026" or "December 2026"). The Admin controls all gates — registration open/closed, application open/closed, and result published — for each session independently.



Phase 1 — Session Setup (Admin)

Admin configures the admission session before any student can register



Session Management

Admin Panel → /sessions

- 1 **Create Session** — Enter year + month (e.g. July 2026 or December 2026). One session per intake.
- 2 **Activate Session** — Only ONE session can be active at a time. All others auto-deactivated on activation.
- 3 **Open Registration** — Toggle `registration_open` = 1. Students can now register.
- 4 **Open Applications** — Toggle `application_open` = 1. Students can submit their forms.
- 5 **Publish Results** — Toggle `result_published` = 1. Automatically forces `application_open` = 0 and unlocks counselling for qualified students.



Community Fees

Admin Panel → /community-fees

- 1 Define fee amounts per community category (OC, BC, MBC, SC, ST)
- 2 Add new community rows, edit amounts inline, delete unused rows
- 3 Fees displayed to students during payment step



Dropdown Master Data

Admin Panel → /dropdowns

- 1 Departments, Programmes, Research Areas
- 2 Religions, Castes, Mediums of Instruction
- 3 Academic qualifications, passing years
- 4 Countries, States, Districts (for address form)



University Settings

Admin Panel → /settings

- 1 **University Info** — Name, logo, address, contact
- 2 **Exam Details** — Exam date, venue, instructions
- 3 **Announcements** — Scrolling ticker text for students
- 4 **Payment Gateway** — Fee amount, gateway credentials



Upload Settings

Admin Panel → /uploads

- 1 Define allowed file types per document (photo, signature, certificates)
- 2 Set max file sizes for each upload field



Phase 2 — Student Registration & Login

Student portal entry — gated by `registration_open` flag on active session



Register

Student Portal → /register

Gate: registration_open = 1

- 1 Check active session — if `registration_open = 0`, form is fully disabled with "Registrations closed" banner
- 2 Student fills: Full Name, Email, Mobile, Date of Birth, Gender, Password
- 3 POST `/api/auth/register` → auto-assigned to active session
- 4 System generates unique Application ID: APP2026-XXXXXX
- 5 Application ID displayed — student must note it for login



Login

Student Portal → /login

- 1 Student enters Application ID + Password
- 2 POST `/api/auth/login` → returns JWT token
- 3 Token stored in Zustand store (in-memory + localStorage)
- 4 Redirect to `/dashboard`
- 5 All subsequent API calls use Bearer token in Authorization header



Phase 3 — Application Form

Multi-section form with auto-save draft and gated final submission



ApplicationForm.jsx

Student Portal → /apply

Gate: application_open = 1 (for final submit only)

① Personal Details

- Full name, date of birth
- Gender, religion, caste
- Community category

② Contact Details

- Correspondence & permanent address
- State, District, Pincode
- Mobile, alternate mobile, email

③ Academic Details

- UG/PG qualifications
- Percentage, passing year
- Institution, university

④ Research Details

- Department & Programme
- Area of Research
- Mode of study (FT/PT)

⑤ Document Upload

- Passport photo, signature
- Degree certificates
- Community & other certificates

⑥ Declaration & Submit

- Checkbox confirmation
- Save Draft (always allowed)
- Final Submit (gated)



Save Draft — POST /api/applications/save with status='Draft'. Always allowed regardless of application_open flag. Student can return and edit at any time.



Final Submit — POST /api/applications/save with status='Submitted'. Blocked with 403 if application_open = 0. Warning banner shown when closed.



Phase 4 — Admin Reviews Applications

Admin views, filters, scores, and updates each student's application



Applications List

Admin Panel → /applications

- 1 View all applications with pagination and search
- 2 Filter by session, year, month, status, payment, qualification
- 3 Export full list to Excel (with all fields)
- 4 Stats dashboard: total, submitted, approved, paid counts
- 5 Add applications manually (walk-in candidates)



Application Detail

Admin Panel → /applications/:id

- 1 View all submitted fields + uploaded documents
- 2 **Update Status:** Draft / Submitted / Approved / Rejected
- 3 **Update Payment Status:** Unpaid / Paid / Failed
- 4 **Mark Attendance:** Present / Absent (post-exam)
- 5 **Enter Entrance Mark + Interview Mark**
- 6 **Set Qualification Status:** Pending / Qualified / Direct Qualified / Failed / Absent
- 7 **Set Admission Status** (final decision)



Phase 5 — Fee Payment

Application fee must be paid to proceed to hall ticket generation



Student Payment

Student Portal → /payment

- 1 Available only after application is Submitted (not Draft)
- 2 Fee amount shown (from community-based fee table)
- 3 Payment via integrated gateway
- 4 Status updates: Unpaid → Paid or Failed
- 5 Dashboard shows green "Payment Confirmed" or orange "Retry Payment" banner



Admin Payment Override

Admin Panel → /applications/:id

- 1 Admin can manually set payment_status to Paid/Unpaid/Failed
- 2 PUT /api/admin/applications/:id/payment-status
- 3 Used for offline/NEFT payments or payment corrections



Phase 6 — Hall Ticket

Admin generates hall tickets for paid students; students download them



Hall Ticket Management

Admin Panel → /hall-tickets

- 1 **Eligible List** — GET /eligible: students with payment=Paid and no ticket yet
- 2 **Generate** — POST /generate: creates hall_ticket_number, assigns exam center & date
- 3 **Issued List** — GET /issued: students who already have a hall ticket
- 4 **Print** — GET /print/:id: printable formatted hall ticket
- 5 **Email** — POST /send/:id: auto-email hall ticket to student
- 6 **Delete** — DELETE /:id: remove and regenerate if needed



Student Hall Ticket

Student Portal → /hall-ticket

- 1 Dashboard shows green "Hall Ticket Ready!" banner when issued
- 2 Student navigates to /hall-ticket page
- 3 Downloads/prints hall ticket PDF with: student photo, exam date, exam center, hall ticket number, exam instructions
- 4 GET /api/student/hall-ticket returns hall ticket data



Phase 7 — Entrance Exam & Results

Physical exam → Admin scores students → Admin publishes results to all students simultaneously



Post-Exam Scoring

Admin Panel → /applications/:id

- 1 Mark attendance for each student: Present / Absent
- 2 Enter entrance exam numeric score
- 3 Enter interview mark (if applicable)
- 4 Set qualification_status: **Qualified / Direct Qualified / Failed / Absent**



Publish Results

Admin Panel → /sessions → Toggle Result

- 1 Admin clicks "Publish Results" for the active session
- 2 PUT /api/sessions/:id/toggle-result
- 3 result_published = 1, application_open auto-forced = 0
- 4 All students immediately see results on their Dashboard
- 5 Counselling unlocked ONLY for Qualified / Direct Qualified
- 6 Admin can unpublish to lock again (result_published = 0)



Qualified / Direct Qualified

Green card shown. "Apply for Counselling" button visible. Counselling gate opens.



Failed

Red card shown. No counselling button.
"Contact university for details."



Absent

Grey card shown. No counselling button.
"Contact university for clarification."



Phase 8 — Counselling

Qualified students choose research preferences; admin reviews and assigns supervisors



Dual Gate — both must pass: (1) result_published = 1 AND (2) qualification_status IN ('Qualified', 'Direct Qualified'). If either fails, student gets a 403 with clear message. Admin sets up centers/supervisors before students apply.



Admin — Counselling Setup

Admin Panel → /counselling

- 1 **Counselling Settings** — Create settings for the session: deadline, instructions, max preferences. POST /api/admin/counselling/settings
- 2 **Research Centers** — Add active research centers. POST /api/admin/counselling/research-centers
- 3 **Research Supervisors** — Add supervisors per center with vacancy count. POST /api/admin/counselling/research-supervisors
- 4 Edit or delete centers/supervisors as needed



Admin — Review Applications

Admin Panel → /counselling/applications

- 1 View all counselling applications submitted by students
- 2 Filter by session, center, supervisor, status
- 3 Review student research preferences (ordered list)
- 4 Assign final supervisor and research center
- 5 Update admission status
- 6 Export to Excel for final allotment



Student — Apply for Counselling

Student Portal → /counselling



Gate 1: result_published = 1



Gate 2: qualification_status = Qualified / Direct Qualified

- 1 Student navigates to counselling via green result card button or sidebar
- 2 System checks both gates (API: GET /api/counselling/settings)
- 3 If gates fail → clear error message shown, no form displayed
- 4 If gates pass → student sees available Research Centers and Supervisors
- 5 Student selects up to N preferences, each with: Center + Supervisor, ranked in order
- 6 **Save Draft** → POST /api/counselling/save (status='Draft')
- 7 **Final Submit** → POST /api/counselling/submit
- 8 Both save and submit enforce both gates on the backend



Student Dashboard — Live Workflow Timeline

Dashboard automatically reflects the student's current position in the 6-stage workflow



Admission Session: May 2026

Session status indicators shown to student in real-time



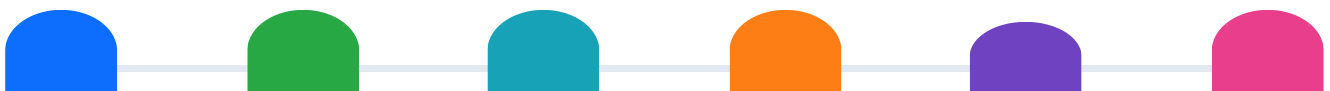
Active

Registration Open

Applications Open



Results Published





Application
Submit your
application form



Fee Payment
Payment
confirmed



Hall Ticket
Issued —
download now



Entrance Exam
Attendance:
Present



Result
 Qualified



Counselling
You are eligible
— apply now



Complete API Reference

All endpoints across both backends (Student: port 5000 | Admin: port 5001)

🎓 Student Backend — localhost:5000/api/

Method	Endpoint	Auth	Description
POST	/auth/register	None	Register new student (gated: registration_open=1)
POST	/auth/login	None	Login with Application ID + password → JWT
GET	/active-session	None	Returns active session with all flags (public)
GET	/student/session	JWT	Student's assigned session details + result_published
GET	/applications/:appId	JWT	Get student's application data (all fields)
POST	/applications/save	JWT	Save/submit application (gated: application_open=1 for submit)
GET	/applications/download-pdf/:appId	JWT	Download application as PDF
GET	/student/hall-ticket	JWT	Get student's hall ticket data
GET	/counselling/settings	JWT	Get counselling settings (gated: result+qualification)
GET	/counselling/research-centers	None	List active research centers
GET	/counselling/research-supervisors	None	Supervisors for a center (?center_id=X)
GET	/counselling/my-application	JWT	Get student's counselling application
POST	/counselling/save	JWT	Save counselling draft (gated: result+qualification)
POST	/counselling/submit	JWT	Final submit counselling application
GET	/settings	None	University-wide settings (public)
GET	/dropdowns/:table	None	Master dropdown data (departments, religions, etc.)
GET	/states / /districts	None	Location data for address form

🔑 Admin Backend — localhost:5001/api/ (all require JWT + isAdmin)

Method	Endpoint	Description
GET	/sessions	All sessions with registered user counts
POST	/sessions	Create new session
PUT	/sessions/:id/activate	Activate session (auto-deactivates all others)
PUT	/sessions/:id/toggle-registration	Open/close student registration
PUT	/sessions/:id/toggle-application	Open/close application submissions
PUT	/sessions/:id/toggle-result	Publish/unpublish results (auto-closes applications on publish)
DEL	/sessions/:id	Delete session (blocked if users assigned)
GET	/applications	All applications (filterable)
GET	/applications/stats	Dashboard counts (total, submitted, paid, etc.)
GET	/applications/export/excel	Export applications to Excel

Method	Endpoint	Description
GET	/applications/:id	Full application detail
PUT	/applications/:id/status	Update application status
PUT	/applications/:id/payment-status	Update payment status
PUT	/applications/:id/attendance	Mark attendance (Present/Absent)
PUT	/applications/:id/entrance-mark	Enter entrance exam score
PUT	/applications/:id/interview-mark	Enter interview score
PUT	/applications/:id/admission	Set final admission status
GET	/hall-tickets/eligible	Students eligible for hall ticket (Paid, no ticket)
POST	/hall-tickets/generate	Generate hall ticket for a student
POST	/hall-tickets/send/:id	Email hall ticket to student
GET	/counselling/settings	Manage counselling settings per session
GET	/counselling/applications	All counselling applications (filterable)



Database Schema

Key tables in rsm_db — MySQL 8+

sessions

id INT AUTO_INCREMENT PK
year INT
month VARCHAR (e.g. "July")
is_active TINYINT (0/1)
registration_open TINYINT (0/1)
application_open TINYINT (0/1)
result_published TINYINT (0/1)
session_name VARCHAR (computed)
created_at, updated_at TIMESTAMP

users

id INT AUTO_INCREMENT PK
application_id VARCHAR UNIQUE (APP2026-XXXXXX)
full_name VARCHAR
email VARCHAR UNIQUE
mobile VARCHAR
password VARCHAR (bcrypt)
date_of_birth DATE
gender VARCHAR
session_id INT FK → sessions.id
created_at TIMESTAMP

applications

id INT AUTO_INCREMENT PK
user_id INT FK → users.id
status ENUM (Draft/Submitted/Approved/Rejected)
payment_status ENUM (Unpaid/Paid/Failed)
attendance_status ENUM (Present/Absent/NULL)
entrance_mark DECIMAL
interview_mark DECIMAL
qualification_status ENUM (Pending/Qualified/Direct
Qualified/Failed/Absent)
admission_status VARCHAR
+ all form fields (personal, contact, academic, research)

hall_tickets

id INT AUTO_INCREMENT PK
user_id INT FK → users.id
session_id INT FK → sessions.id
hall_ticket_number VARCHAR UNIQUE
exam_center VARCHAR
exam_date DATE
exam_time TIME
created_at TIMESTAMP

counselling_applications

id INT AUTO_INCREMENT PK
user_id INT FK → users.id
session_id INT FK → sessions.id
status ENUM (Draft/Submitted)
created_at, updated_at TIMESTAMP

counselling_research_choices

id INT AUTO_INCREMENT PK
counselling_application_id INT FK
center_id INT FK → research_centers.id
supervisor_id INT FK → research_supervisors.id
preference_order INT (1, 2, 3...)

counselling_settings

id INT AUTO_INCREMENT PK
session_id INT FK → sessions.id
deadline DATETIME
instructions TEXT
max_preferences INT
is_active TINYINT

community_fees

id INT AUTO_INCREMENT PK
community VARCHAR (OC, BC, MBC, SC, ST)
fee_amount DECIMAL(10,2)
created_at, updated_at TIMESTAMP



Permission & Gate Matrix

Every action and the exact condition required for it to succeed

Who	Action	Required Condition	On Failure
Student	Register	Active session exists AND registration_open = 1	Form disabled, 🚫 banner shown
Student	Login	Valid Application ID + correct password	401 Invalid credentials
Student	Save Draft	Authenticated (JWT valid)	401 Token invalid/missing
Student	Final Submit Application	JWT valid AND application_open = 1	403 "Submissions closed"
Student	View Hall Ticket	hall_ticket_number exists for student	Empty state "Not issued yet"
Student	See Results	result_published = 1 on student's session	"Results not yet published" shown
Student	Access Counselling	result_published = 1 AND qualification_status IN (Qualified, Direct Qualified)	403 with specific reason message
Student	Submit Counselling	Same dual gate enforced server-side again	403 "Not eligible" or "Results not published"
Admin	All Admin APIs	Valid JWT AND isAdmin middleware passes	401 / 403 Access denied
Admin	Activate Session	Always allowed — auto-deactivates all others	—
Admin	Publish Results	Always allowed — auto-closes application_open	—
Admin	Delete Session	No users (students) assigned to this session	409 "Cannot delete: N users belong to this session"